

Name : Fahad Ali

Roll no : 23P-0569

Section : CS-4K

Lab Task:

Task:1

C code for pipe:

```
New Open... Save Save As... Undo Redo
main.c x Task1.c x
2 #include <unistd.h>
3 #include <string.h>
4 #include <ctype.h>
5
6 int main() {
7     int fd[2];
8     char str[] = "HELLO FROM FAHAD"; //send data in capital string child convert it in lower letter.....
9     char buffer[100];
10
11     if (pipe(fd) == -1) {
12         perror("pipe");
13         return 1;
14     }
15
16     pid_t pid = fork();
17     if (pid < 0) {
18         perror("fork failed...");
19         return 1;
20     }
21
22     if (pid > 0) { // if true this is Parent process...
23
24         close(fd[0]); //now close read end.....
25         write(fd[1], str, strlen(str) + 1);
26         close(fd[1]);
27     }
28     else //this is a childe proces.....
29     {
30         close(fd[1]);
31         read(fd[0], buffer, sizeof(buffer));
32
33         for (int i = 0; buffer[i]; i++)
34             buffer[i] = tolower(buffer[i]); //converting into lower.....
35         printf("Child received Capital string [%s] and converted into lower: %s\n", str, buffer);
36         close(fd[0]);
37     }
38     return 0;
39 }
40
```

Output:

```
fkhokar@fedora:~/Desktop/0sTask-10$ gcc Task1.c
fkhokar@fedora:~/Desktop/0sTask-10$ ./a.out
Child received Capital string [HELLO FROM FAHAD] and converted into lower: hello from fahad
fkhokar@fedora:~/Desktop/0sTask-10$
```

Task:2

code

```
Task1.c x task2.c x
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/mman.h>
5  #include <fcntl.h>
6  #include <semaphore.h>
7
8  typedef struct {
9      double balance;
10     sem_t mutex;
11 } SharedBank;          //this is act lik shared memory.....
12
13 int main() {
14     int fd = shm_open("/bank_shm", O_CREAT | O_RDWR, 0666);
15     ftruncate(fd, sizeof(SharedBank));
16
17     SharedBank *bank = mmap(NULL, sizeof(SharedBank), PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
18
19     sem_init(&bank->mutex, 1, 1); //semaphore ki instize krhai yaha...
20
21     if (fork() == 0)
22     {
23         sem_wait(&bank->mutex);
24         bank->balance += 100.0;
25         printf("Child updated the balance: %.2f\n", bank->balance);
26         sem_post(&bank->mutex);
27         exit(0);
28     } else {
29         sleep(1);
30         sem_wait(&bank->mutex);
31         printf("Parent reads balance: %.2f\n", bank->balance);
32         sem_post(&bank->mutex);
33     }
34
35     sem_destroy(&bank->mutex); //distroying semapohore.....
36     munmap(bank, sizeof(SharedBank));
37     shm_unlink("/bank_shm");
38     return 0;
39 }
40
```

Output:

```
File Edit View Bookmarks Plugins Settings Help
+ New Tab Split View Copy Paste
fkhokar@fedora:~/Desktop/0sTask-10$ gcc task2.c
fkhokar@fedora:~/Desktop/0sTask-10$ ./a.out
Child updated the balance: 100.00
Parent reads balance: 100.00
fkhokar@fedora:~/Desktop/0sTask-10$
```

Data consistency:

Both processes one is updating, one is reading share the same memory for the balance. To make sure they don't change or read the balance at the same time, we use a semaphore.

The semaphore acts like a lock A process locks the balance before using it. After finishing, it unlocks it. This way, only one process can access the balance at a time, so the balance always stays correct and consistent.

Task:3 code

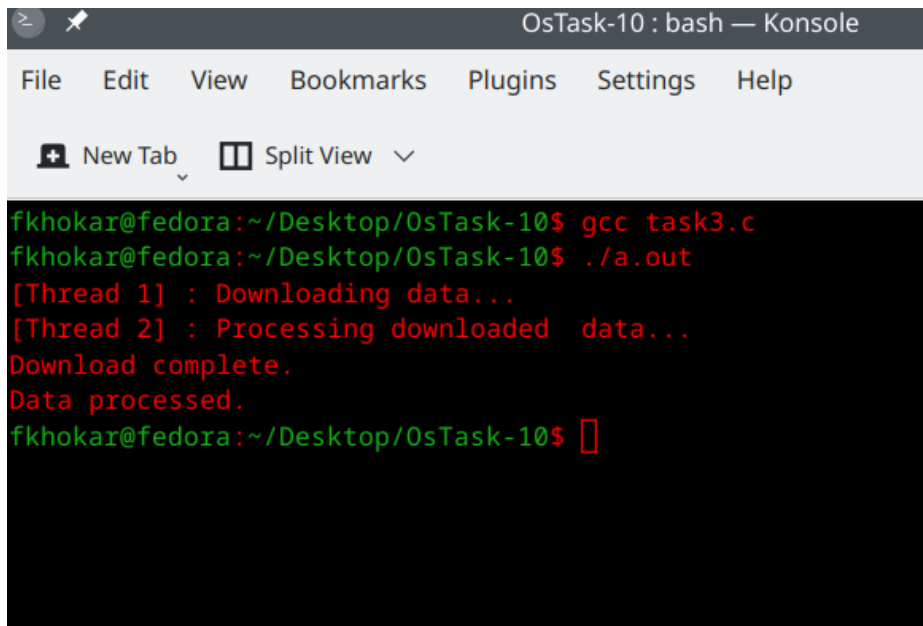
```
Task1.c x task2.c x task3.c x
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

void* download(void* arg) {
    printf("[Thread 1] : Downloading data...\n");
    sleep(1);
    printf("Download complete.\n");
    return NULL;
}

void* process(void* arg) {
    printf("[Thread 2] : Processing downloaded data...\n");
    sleep(3);
    printf("Data processed.\n");
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, download, NULL);
    pthread_create(&t2, NULL, process, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    return 0;
}
```

Output:



A screenshot of a terminal window titled "OsTask-10 : bash — Konsole". The window has a menu bar with "File", "Edit", "View", "Bookmarks", "Plugins", "Settings", and "Help". Below the menu bar is a toolbar with "New Tab" and "Split View" options. The terminal content shows a user named "fkhokar" at a "fedora" machine in the directory "~/Desktop/OsTask-10". The user runs the command "gcc task3.c", followed by "./a.out". The program outputs two lines: "[Thread 1] : Downloading data..." and "[Thread 2] : Processing downloaded data...". It then prints "Download complete." and "Data processed." before returning to the prompt. A red cursor is visible at the end of the final prompt line.

```
OsTask-10 : bash — Konsole
File Edit View Bookmarks Plugins Settings Help
+ New Tab Split View
fkhokar@fedora:~/Desktop/OsTask-10$ gcc task3.c
fkhokar@fedora:~/Desktop/OsTask-10$ ./a.out
[Thread 1] : Downloading data...
[Thread 2] : Processing downloaded data...
Download complete.
Data processed.
fkhokar@fedora:~/Desktop/OsTask-10$
```

Task:4



The image shows a code editor window with a dark theme. The title bar at the top includes menu items: New, Open..., Save, Save As..., Undo, and Redo. Below the title bar, there are four tabs: Task1.c, task2.c, task3.c, and task4.c. The task4.c tab is active. The code is written in C and is as follows:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <signal.h>
5  #include <sys/wait.h>
6
7  //handler for Ctrl+C
8  void handle_sigint(int sig) {
9      printf("\nCaught SIGINT (Ctrl+C). Press again to exit.\n");
10 }
11
12
13 void handle_sigchld(int sig) {
14     int status;
15     pid_t pid;
16     while ((pid = waitpid(-1, &status, WNOHANG)) > 0) {
17         printf("Child process %d terminated.\n", pid);
18     }
19 }
20
21 int main() {
22
23     signal(SIGINT, handle_sigint);    // Ctrl C
24     signal(SIGCHLD, handle_sigchld); //ye child termination handle krai ga....
25
26     pid_t pid = fork();
27
28     if (pid < 0) {
29         perror("fork failed");
30         exit(1);
31     }
32     else if (pid == 0) {
33
34         printf("Child process PID: %d\n", getpid());
35         printf("Child sleeping for 10 seconds...\n");
36         sleep(10);
37         printf("Child done.\n");
38         exit(0);
39     }
```

```

10     else {
11
12         printf("Parent process PID: %d\n", getpid());
13         printf("Parent waiting. Try pressing Ctrl+C...\n");
14
15         // running loop until child finishes.....
16         for (int i = 0; i < 15; i++) {
17             printf("Parent running... %d\n", i);
18             sleep(1);
19
20             if (i == 5) {
21
22                 printf("Parent sending SIGKILL to child %d\n", pid);
23                 kill(pid, SIGKILL);
24             }
25         }
26         printf("Parent exiting.\n");
27     }
28
29     return 0;
30 }
31

```

Output and demonstration:

Starting of program....

```

File Edit View Bookmarks Plugins Settings Help
+ New Tab Split View Copy
fkhokar@fedora:~/Desktop/OsTask-10$ gcc task4.c
fkhokar@fedora:~/Desktop/OsTask-10$ ./a.out
Parent process PID: 27919
Parent waiting. Try pressing Ctrl+C...
Parent running... 0
Child process PID: 27920
Child sleeping for 10 seconds...
Parent running... 1
Parent running... 2
Parent running... 3

```

Pressing CTRL C....

Instead of the program terminating this will show
This is showing the custom SIGINT handler is working

```
Parent running... 1
Parent running... 2
Parent running... 3
Parent running... 4
^C

Caught SIGINT (Ctrl+C). Press again to exit.
Caught SIGINT (Ctrl+C). Press again to exit.
Child done.
Parent running... 5
Child process 28080 terminated.
Parent sending SIGKILL to child 28080
Parent running... 6
Parent running... 7
□
```

Forceful terminationn of child when i=5

Even if the child was sleeping, SIGKILL terminates it immediately. The parent's SIGCHLD handler prints that the child has terminated. Showing in below ss highlighted

```
Caught SIGINT (Ctrl+C). Press again to exit.
Caught SIGINT (Ctrl+C). Press again to exit.
Child done.
Parent running... 5
Child process 28080 terminated.
Parent sending SIGKILL to child 28080
Parent running... 6
Parent running... 7
Parent running... 8
Parent running... 9
Parent running... 10
Parent running... 11
```

Then programme excute and end.....

```
Parent sending SIGKILL to child 28080
Parent running... 6
Parent running... 7
Parent running... 8
Parent running... 9
Parent running... 10
Parent running... 11
Parent running... 12
Parent running... 13
Parent running... 14
Parent exiting.
fkhokar@fedora:~/Desktop/0sTask-10$
```

Task:5

code

```
Task1.c x task2.c x task3.c x task4.c x task5.c x
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_USERS 5

pthread_mutex_t printer_mutex;

void* print_job(void* arg) {
    int user_id = *(int*)arg;

    printf("[User %d] wants to print\n", user_id);

    pthread_mutex_lock(&printer_mutex);
    printf("[User %d] is using the printer\n", user_id);

    sleep(2);

    printf("[User %d] finished printing\n", user_id);

    pthread_mutex_unlock(&printer_mutex);

    return NULL;
}
```

```

int main() {
    pthread_t users[NUM_USERS];
    int user_ids[NUM_USERS];

    pthread_mutex_init(&printer_mutex, NULL);

    for(int i = 0; i < NUM_USERS; i++) {
        user_ids[i] = i + 1;
        pthread_create(&users[i], NULL, print_job, &user_ids[i]);
    }

    for(int i = 0; i < NUM_USERS; i++) {
        pthread_join(users[i], NULL);
    }

    pthread_mutex_destroy(&printer_mutex);

    return 0;
}

```

Output:

```

OsTask-10: bash — Konsole
File Edit View Bookmarks Plugins Settings Help
New Tab Split View Copy Paste
fkhokar@fedora:~/Desktop/OsTask-10$ gcc task5.c
fkhokar@fedora:~/Desktop/OsTask-10$ ./a.out
[User 1] wants to print
[User 1] is using the printer
[User 3] wants to print
[User 4] wants to print
[User 2] wants to print
[User 5] wants to print
[User 1] finished printing
[User 3] is using the printer
[User 3] finished printing
[User 4] is using the printer
[User 4] finished printing
[User 2] is using the printer
[User 2] finished printing
[User 5] is using the printer
[User 5] finished printing
fkhokar@fedora:~/Desktop/OsTask-10$

```

Explanation:

A race condition happens when two or more users access a shared resource at the same time, causing unpredictable results

Synchronization using a mutex prevents this by allowing only one user to access the resource at a time, so operations happen in order and data stays safe